Faculty of Business & Technology

School of Computer Science & Engineering

CET2007 Assignment : Functional Application Design and Development

---

This assessment contributes 100% to your final module mark. You are expected to spend approximately 50 hours completing this assessment.

Learning Outcomes

The following learning outcomes will be assessed:

- Apply advanced object-oriented programming constructs to develop professional-standard software.
- Select and apply software design patterns, data structures, and algorithms to solve complex programming problems
- Design, develop, and test software artefacts using industry-standard techniques and practices.
- Use version control, testing, and documentation tools to support software development

Important Information

You are required to submit your work within the bounds of the University Infringement of Assessment Regulations (see your Programme Guide). Plagiarism, paraphrasing and downloading large amounts of information from external sources, will not be tolerated and will be dealt with severely. Although you should make full use of any source material, which would normally be an occasional sentence and/or paragraph (referenced) followed by your own critical analysis/evaluation. You will receive no marks for work that is not your own. Your work may be subject to checks for originality, which can include use of an electronic plagiarism detection service.

Where you are asked to submit an individual piece of work, the work must be entirely your own. The safety of your assessments is your responsibility. You must not permit another student access to your work. Where referencing is required, unless otherwise stated, the Harvard referencing system must be used.

Please ensure that you retain a duplicate of your assignment. We are required to send samples of student work to the external examiners for moderation purposes. It will also safeguard in the unlikely event of your work going astray.

Submission Date and Time	As specified on Canvas
Submission Location	submission via Canvas

➢ Introduction

You are a junior developer at a small software consultancy. A client has approached your team and requested a **console-based management system in C#**. The system must allow users to add, update, search, sort, and persist data. You will implement your system according to best practices in software engineering, including OOP design, UML diagrams, design patterns, exception handling, algorithms, testing, and documentation.

The client has provided **three possible project scenarios**. You must select **one** to implement. Each scenario requires the same technical skills, but the context is different depending on the client's needs.

Scenario 1: Game Library & Player Stats Manager

A small indie game studio has requested a **Game Library and Player Statistics Manager** to track players and their gaming performance. The system should allow new players to be added, record their gameplay statistics (such as hours played and high scores), and update these stats over time. Players should be searchable by ID or username, and reports should be generated to display the most active players or top scores. Data must be saved and loaded using files so that progress is not lost, and important actions should be logged.

Scenario 2: IoT Device Monitor

A client in the networking sector has requested an **IoT Device Monitoring System** to manage a list of devices across their infrastructure. The system should allow new devices to be added with a unique ID, name, and IP address, while preventing duplicates. Device statuses (online, offline, maintenance) must be updated as required, and users should be able to view, search, and sort devices easily. Data must be persisted to files and exported in report form to summarise system health. Key events should also be logged for auditing purposes.

Scenario 3: Task & Project Tracker

A software company has requested a **Task and Project Tracking System** to help small teams organise their work. The system should allow users to create tasks with unique IDs, due dates, priorities, and assignees. Task statuses (to-do, in progress, done) must be updated as work progresses, and the system should allow users to search tasks and sort them by priority or deadline. Data must be stored in files so tasks are not lost between sessions, and the system should be able to export a report highlighting overdue or upcoming tasks. Logs should record important actions such as task creation or status updates.

➢ Assignment Specifications

This assignment requires you to choose one of the three scenarios above and build a working application using C#. You will be assessed on both your theoretical understanding and practical application of programming, design and testing concepts. This will include demonstrating your

ability to utilize object-oriented principles, manage data structures, and handle file input/output, which are all key parts of this module.

Task 1 : Design & Documentation (20 marks)

you are required to produce appropriate design documentation and maintain supporting project records. This stage reflects professional practice in system design and software development, where planning and documentation guide implementation. Include screenshots of all the parts.

- **UML Diagrams**
  - A UML Class Diagram showing the main classes, their relationships, and use of inheritance/interfaces.
  - At least one Sequence Diagram illustrating a core system process (e.g., adding a device, recording stats, or creating a task).
- **Version Control & Documentation**
  - A GitHub repository containing your project, with regular commits.
  - A README.md (or GitHub Wiki) explaining the system purpose, how to run it, and documenting design choices.
- **Agile Practices**
  - Use a simple Agile board (Trello, or similar) to track progress. Screenshots should be included in your design document.

Task 2 : Software Implementation (60 marks)

You must implement your chosen scenario in C# as a console application. Your application should be fully functional and make use of the following programming and design concepts:

- **Object-Oriented Principles**
  - → Classes, inheritance, interfaces, and polymorphism.
  - → Encapsulation of functionality (e.g., keep Program.cs minimal).
- **Data Structures & Persistence**
  - → Use of **collections and generics** to manage system records.
  - → Data persistence using **JSON**.
  - → **Logs** must be written to a TXT file.
  - → Robust **file I/O** with error handling (missing file, malformed data).
- **Algorithms**
  - → At least one **search** (linear or binary where applicable).
  - → At least one **sorting algorithm** (basic manual implementation plus/or use of built-in sort for comparison).
  - → Algorithms must be demonstrated on real system data (e.g., sorting tasks by due date, searching players by ID).
- **Error Handling**
  - → Implement appropriate **exception handling** to ensure the program is robust against invalid input and file errors.

- **Design Patterns**
  → Apply **design patterns**. You will explain and justify your chosen patterns in your **screencast video**.

Task 3 : Testing (10 marks)

Testing is a key part of professional software development and must be integrated into your project.

- Create a **unit test project** and implement at least **five unit tests** to validate individual methods and classes in your system. Include screenshots in the design document.
- Include a short **Debugging Reflection** in your design document describing at least one bug you encountered. Explain how you used Visual Studio's debugging tools (e.g., breakpoints, watch window, call stack) to identify the problem. Then describe the fix you applied and confirm how you verified it worked. Provide one or two relevant **screenshots** in the design document to show your use of Visual Studio's debugging tools.

Task 4 : Screencast Demonstration (10 marks)

To complete your submission, you will create a short screencast video of no more than 10 minutes. This video should demonstrate your application in action. You will walk through the main menu, show each of the key functionalities working correctly, and explain how the application save and load data, search and sort functions. You should also run your unit tests and lastly briefly explain the selected design patterns and briefly justify why you selected them.

➢ Submission Requirements

Submit all completed tasks through the Assignment submission link on Canvas. Following the new requirements for assignment submissions, you are required to fill out the academic declaration form available on Canvas Assignments tab, declaring any use of AI in the assignment.

Generative AI may be used only to support your learning and understanding of concepts. Any use of Generative AI or automated tools to generate parts of this assignment (e.g., code, diagrams, recorded screencast or written sections) will result in 0 marks for each affected element and may be referred for an academic misconduct investigation.

This assignment should be submitted no later than the date advised on Canvas, including:

- **Academic Declaration Form** : complete the academic declaration form on Canvas. Please ensure this form is completed. Failure to do so may result in your assignment submission not being accepted.

- **Design Document** : Submit a single document (PDF or Word) containing your UML diagrams, Agile board screenshots, GitHub repository link and screenshots, and your Debugging Reflection (with unit test screenshot).

- **Source Code**: Submit a zipped folder containing your full Visual Studio solution. This must include .sln file with all .cs source files, your **main application and unit test project**, and any data files (JSON, TXT logs, CSV reports). Ensure the project runs without missing files.

- **Screencast Video** : Submit a video of your screencast demonstration, adhering to the specified duration of ~10 minutes. Ensure that the screencast effectively covers all aspects of your application and tests, as outlined in the tasks description. The video should be clear, well-paced, and professionally presented, with visible on-screen content. Use appropriate recording software such as OBS (or another screen recording tool of your choice), and submit the screencast in a commonly used and accessible file format (MP4).

➢ Marking Scheme

The mark scheme and breakdown is shown below.

Criteria	Excellent (80%-100%)	Good (60%-80%)	Fair (40%-60%)	Poor (20%-40%)	Unsatisfactory (0%-20%)
Design & Documentation (20 marks)	Comprehensive UML with correct notation; Agile board realistic and regularly updated; GitHub repo with consistent commit history; Debugging Reflection clear with screenshot evidence.	UML mostly correct with minor errors; Agile board functional but less detailed; GitHub repo with some commit history; Debugging Reflection explains bug but limited depth.	UML diagrams incomplete or simplistic; Agile board inconsistent; GitHub repo shows minimal commits; Debugging Reflection vague.	UML incorrect or very limited; Agile board superficial; GitHub repo rarely used; Debugging Reflection very weak.	No or very poor UML; no or very poor Agile evidence; no or very poor GitHub history; no or very poor Debugging Reflection.
Software Implementation: OOP & Design Patterns **(20 marks)**	Advanced OOP: inheritance, polymorphism, encapsulation, interfaces; design patterns applied correctly and justified; modular architecture.	OOP used well with inheritance/polymorphism; one design pattern applied effectively; generally modular.	Limited OOP; weak or incomplete pattern use; some modularity but inconsistent.	Minimal OOP beyond basic classes; no correct design pattern use; code structure weak.	No or very poor evidence of OOP or design patterns.
Software Implementation: Data Structures, Persistence & File I/O **(20 marks)**	Effective use of collections/generics; JSON persistence robust; TXT logs detailed and consistent; strong error handling.	Collections used appropriately; JSON persistence works with minor issues; TXT logs present but limited; some error handling.	Basic collections; persistence partly working; logs minimal; weak error handling.	Minimal use of collections; persistence unreliable or missing; logs superficial; poor error handling.	No or very poor collections; no or very poor persistence; no or very poor logging.
Software Implementation: Algorithms, Error Handling & Code Quality **(20 marks)**	One or more search and sort implemented correctly on real data; strong exception handling; clean, readable, modular, well-commented code.	Search/sort implemented but not fully efficient or correct; exception handling present with minor flaws; code mostly readable.	Simplistic or incomplete search/sort; weak exception handling;inconsistent readability.	Attempted search/sort but incorrect; little error handling; poor code quality.	No or very poor search/sort; no or very poor error handling; code unreadable or non-functional.
Testing (10 marks)	5 or more meaningful unit tests with high coverage; all tests pass; Debugging Reflection explains bug clearly with screenshots of debugging tools.	3 or more unit tests; most pass; Debugging Reflection explains bug and fix with some evidence.	1–2 weak tests; Debugging Reflection vague or missing screenshots.	very limited working unit tests; Debugging Reflection incomplete.	No or very poor tests; no or very poor Debugging Reflection.
Screencast (10 marks)	Excellent detailed demonstration of all required functionality.	Very good demonstration of all required functionality.	Good demonstration of most functionality.	Satisfactory though limited demonstration of some functionality.	Very limited demonstration, which much functionality not shown